

present in the list, you have to walk through all elements in the list before you can say for sure that the element is not present in the list. Hence, the complexity of search on a linked list is $O(N)$. For a hash table, assuming no collisions, the complexity is $O(1)$. For a binary search tree (which we assume to be balanced), it is $O(\log N)$. For a binary heap and a d-ary heap, it is $O(N)$. I leave it to the reader to figure out why it is $O(N)$ for a heap.

(5) Consider the following code snippet:

```
int find_fib (int N)
{
    assert (N > 0);
    int f1 = 1;
    int f2 = 1;

    if (N < 2)
        return 1;

    return (find_fib(N-1) + find_fib(N-2));
}
```

Given that *find_fib* is called from the main with *N* as 25, how many total calls to *find_fib* occur?

A call to *find_fib(N)* ends up calling *find_fib(N-1)* and *find_fib(N-2)*. Note that a call to *find_fib(N-1)* will end up calling *find_fib(N-2)*. Hence we find that *find_fib(N-1)* gets called once, *find_fib(N-2)* gets called two times, *find_fib(N-3)* gets called once in *find_fib(N-1)*, once each by every one of the two calls to *find_fib(N-2)* and hence *find_fib(N-3)* gets called three times in total. We can write a recurrence relation to the number of calls to *find_fib* as:

$$C(N) = C(N-1) + C(N-2) + 1$$

...where $C(N)$ is the number of calls to *find_fib* and $C(0) = C(1) = 1$. I leave it to the reader to solve the recurrence and work out the value for $N = 25$.

Sorting/searching/string manipulation problems

(6) You are given an array of $2n+1$ integers and told that except for only one element, all other elements have a duplicate in the array. Can you find the one element that has no duplicate in the array? What is the time complexity of your solution?

While a brute force solution would be to sort the array and perform a linear scan to find the duplicate, there is a much quicker solution. Remember that exclusive ORing of an element 'A' with itself is 0. Since all elements except one have a matching duplicate, just do a XOR of all elements. The unmatched element will be the result.

(7) You are given an array *A* of *N* integers. A majority element *M* appears more than $N/2$ times in the array. For instance, given the array of integers, 10, 6, 10, 10, the majority element is 10, 6, 10, 10. Given the array 10, 6, 10, 3, 10, 21, 5, there is no majority element. For the given array of *N* integers, write an algorithm to find whether a majority element exists and if so, what it is. The algorithm should have a worst-case complexity of $O(N)$.

While at first glance it looks like sorting the array and doing a linear scan to find the majority element would be the right approach, this solution will have a complexity of $O(N \log N)$, which is larger than the $O(N)$ required. Hence we need to devise a more clever approach.

There is a well-known algorithm that achieves the required $O(N)$ complexity by approaching the problem in two steps. The first step is to call the function *find_possible_majority_element*, which returns an array element that could possibly be the majority element in the given array *A*. The algorithm *find_possible_majority_element* returns an element from the given array *A*. In the second step, the algorithm *CheckIfMajorityElement(int elem)* actually finds out whether the element *elem* returned by *find_possible_majority_element* in the first step is indeed the majority in *A* by counting the number of occurrences of *elem* in array *A*.

The function *find_possible_majority_element* takes the first element of the array *A* and puts a score 1, and considers it as the current majority element, *elem*. It then takes this element, *elem*, and the current score of 1 as the basis and traverses through the other elements of the array, performing the following checks:

- If the next element in the array is equal to *elem*, then the score is incremented by 1.
- If the next element in the array is not equal to *elem*, then the score is decremented by 1.
- If at the next element, the current score is equal to zero, then the score is set to 1 and this element is considered as *elem*.
- After traversing all elements, finally *find_possible_majority_element* returns *elem* as the possible majority element.

In the next step, *CheckIfMajorityElement* takes this element *elem* as the input argument and counts the number of occurrences of the element in the array *A*. If the count is greater than $n/2$, this element is the majority element; otherwise, null is returned. I leave this as an exercise for readers to write the actual code for the solution.

We will discuss the solutions to the remaining problems in our next column. Meanwhile, I would like to request readers to send in their solutions to these questions. Also, if you are preparing for computer science programming interviews, you may find it useful to visit my programming interviews discussion group "Computer Science Interview Training (India)" on LinkedIn.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me along with your solutions and feedback at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.